

Continuous Model-Driven Engineering

➔ **Tiziana Margaria**, *Potsdam University, Germany*

➔ **Bernhard Steffen**, *TU Dortmund University, Germany*



Agility at the customer, user, and application level will prove key to aligning and linking business and IT.

Agility has become an increasing necessity for application software, particularly when business-critical issues are concerned. Software systems drive and coordinate a company's approach to its product planning, production, marketing, and sales—as well as how it relates to its ecosystem of customers and suppliers.

STRATEGIC IMPACT

These systems are gaining strategic importance, embodying an increasing portion of a company's distinctive know-how and culture. At the same time, they must be continuously updated and adapted to meet the ever-changing market conditions and the inevitable technological change, often driven by globalization, that forces integration and harmonization.

Additional factors complicating this process include increased intermediation in software production, leading to the widespread use of off-the-shelf components and software, outsourcing, and the adoption of services such as X-as-a-service, in which X can be various factors such as software, hosting, and accounting. Other factors include decreased intermediation in the hierarchy of responsibility,

such as making CEOs and CTOs/CIOs personally liable for mishaps such as compliance and governance-level mistakes. These are typically rooted in the software systems that run the company and its communication with its operational environment and ecosystems.

Binding oversight

On the one hand, software production, maintenance, and evolution become increasingly distributed and delegated to third parties. On the other hand, companies feel a growing need to tightly control their own business and its strategies, processes, rules, and regulations. This requires application and business experts to continuously oversee and steer the scenario from their unique perspective. State-of-the-art methods and technologies offer only partial support to bridge this gap:

- Model-driven design shifts the attention and the design activities from the programming level to the modeling level, but still remains in the IT realm. Even at the platform-independent level, the typically UML-based MDD and model-driven architecture approaches provide varieties of model structures that focus

on different technical issues/aspects. Mastering this technology is a special art requiring both IT knowledge and a good sense of dealing with abstraction.

- Aspect orientation organizes/modularizes the realization of different cross-cutting concerns. This not only supports the reuse of the so-called aspect code, but also helps to deal with each aspect delegating the correct positioning to a separately designed weaving mechanism. Aspect orientation is a powerful programming paradigm, particularly for highly distributed implementations to cover, for example, factors such as quality of service, exception handling, or even roles, rights, and permissions.
- eXtreme programming directly aims at continuously involving customer and application experts via frequent prototypes and the “test first” paradigm: Functionalities are prioritized and added one by one, giving the customer the opportunity to review them individually and control the progress. Unfortunately, this approach does not scale, failing particularly when the focus for typical business

applications should be on their global interplay.

- Service-oriented approaches try to overcome the typical problems of traditional “programming in the large” by abandoning the construction of complex monolithic applications for suites of services or service-like artifacts programmed in the small. This supports agility but addresses neither the problem of orchestration nor the management of evolution.
- Process modeling provides a formal basis for application and business experts to formulate their intents unambiguously. This works quite well in the rare cases where the business expert fully knows what he needs and knows how to express it in process notation. Otherwise, the visual nature of process diagrams can easily suggest wrong safety or agreement understandings among stakeholders, while even totally senseless process descriptions often survive lengthy review meetings. Thus, they are typically taken as “better” forms of documentation to be “correctly” interpreted by the IT personnel.

All these approaches contribute to application and business experts continuously overseeing and steering the entire business scenario from their own perspective—but they succeed only partially.

Modeling holistically

What these leaders lack is a holistic view that joins the pieces together, transparently and consistently. Continuous model-driven engineering (CMDE) adopts exactly this view by putting the consistent dialogue of all involved stakeholders into its center, to provide the following:

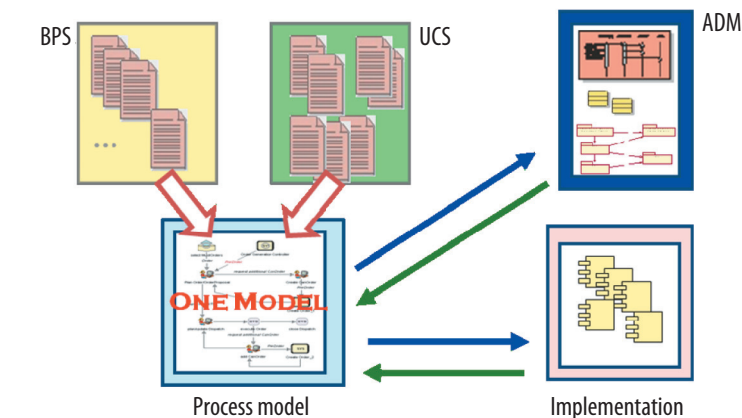


Figure 1. Achieving consistency by aggregating the heterogeneous model landscape of the Rational Unified Process to the “one thing” of CMDE.

- application-level control—the continuous involvement of the customer and application expert along the entire systems’ life cycle, including software maintenance and evolution;
- continuous and ongoing quality assurance with different means at different levels and phases: requirement validation, simulation, model checking, data flow analysis, testing, and monitoring; and
- specific service-oriented support for integrating new technologies easily and noninvasively.

Conceptually, CMDE borrows ideas from the above-mentioned five software and system development paradigms. The combination of eXtreme programming, model-driven design, and process modeling forms the backbone of what we call eXtreme model-driven design (XMDD; T. Margaria and B. Steffen, “Agile IT: Thinking in User-Centric Models,” *Proc. ISoLA 2008*, Springer, 2008, pp. 493-505; T. Margaria and B. Steffen, “Business Process Modeling in the jABC: The One-Thing Approach,” *Handbook of Research on Business Process Modeling*, IGI Global, 2009). This approach is based on the elaboration and refinement of abstractly executable application and business models, called application logic models.

User-centric perspective

XMDD’s user-centric perspective derives from process modeling. Its analysis, verification, and refinement techniques resemble those used in model-driven design, while its devotion to a direct and continuous user experience via model execution provides an eXtreme flavor. Depending on the development stage, execution might mean a simple step-through in terms of if-then games, document browsing, simulation, full execution, or mixtures thereof.

In XMDD, developers continue elaboration and refinement until they reach a predetermined case-specific level, at which point the classical requirement-implementation gap becomes a service-oriented realization of user- and application-level functions. Thus, rather than build highly complex software architectures, XMDD manages complex hierarchical models that orchestrate user- and application-level functions in an agile, bazaar-like fashion, eliminating or substituting rigid cathedral-style approaches (<http://catb.org/~esr/writings/cathedral-bazaar>).

One-thing approach

CMDE now solidifies this perspective via the One-Thing Approach OTA; B. Steffen and P. Narayan, “Full Life-Cycle Support for End-to-End

Processes, *Computer*, Nov. 2007, pp. 64-73), which combines the simplicity of waterfall development with maximum agility. Key to OTA is viewing the entire development process as a cooperative hierarchical and interactive decision process, which is organized by building and refining one comprehensive model, the “one thing.” Figure 1 shows this idea by contrasting it with the Rational Unified Process: Textual process descriptions, use-case diagrams, and other UML diagrams are all aggregated in one single model reflecting their imposed constraints on the eventual solution, which in particular automatically reveals inconsistencies and misunderstandings. With this approach, each stakeholder—including the application expert—can continuously place decisions in terms of constraints, and each development step or evolution can be regarded simply as a transformation or reification of this constraint set. Typical constraints, which might comprise many aspects and touch on several forms of knowledge, are adequately expressed as follows:

- temporal formulae that express the application’s intentions, internal policies, legal constraints, or technical frame conditions;
- loose process models, which specify the rough distributed workflow from the management perspective, without concern for technicalities like type correctness, location, or interoperability that are taken care of elsewhere at a more technical level;
- symbolic type information, sufficient to imply executability, and enforced later by synthesis technology; and
- definitions of roles and rights, timing and localization constraints, and exception handling, which will be integrated during aspect-oriented code generation.

In this view, the development process’s waterfall character focuses on

the chosen decision hierarchy: who can decide or modify what, the binding power assigned to decisions, and how conflicts should be resolved. This approach lets all the stakeholders monitor globally and at any time the development or evolution process via constraint checking, along the structure of knowledge and responsibility. At the same time, it imposes a kind of decision hierarchy by mapping areas of competencies to individuals’ roles to identify required actions and avoid constraint violation.

This new view of cooperation between the customer and the contractor/developer reduces the need for complex and expensive specification documents.

Like XP for programming in the small, this approach revolutionizes process and application development by replacing the typically long intervals between contract-and-requirements time and delivery-and-acceptance time—with all the associated pitfalls—by a continuous, cooperative development process that reveals misconceptions so they can be addressed as soon as they arise.

Understanding of the application under construction—the user experience—builds naturally along the way. In particular, this new view of cooperation between the customer and the contractor/developer reduces the need for complex and expensive specification documents. These all too often impose design decisions before the customer can judge their impact, and they often turn out to be obstacles when adjusting the project in the overall context. In contrast, CMDE, with its cooperative development and evolutionary style, supports the agile adjustment by focusing on the following points:

- *Continuously update the customer.* The impact of each design decision on the application logic becomes apparent via OTA, which provides customers with a continuously updated user experience.
- *Focus on the application logic.* This lets developers repair and modify at the level where the need appears, while following the service-oriented paradigm makes noninvasively exchanging or integrating third-party functionality easy.
- *CMDE, and particularly OTA.* These methods reveal their power in the context of formal methods-based tooling with jABC, our corresponding development and execution framework (B. Steffen et al., “Model-Driven Development with the jABC,” *Proc. Haifa Verification Conf.*, Springer, LNCS 4383, 2006, pp. 92-101; M. Hörmann et al., “The jABC Approach to Rigorous Collaborative Development of SCM Applications,” *Proc. ISoLA 08*, Springer Verlag, 2008, pp. 724-737), which provides application-level tracing and simulation models executable in a mode reflecting the current development stage.
- *Constraint analysis.* At any stage, developers can verify the formulated constraints via local checking and data-flow analysis to ensure parameterization and configuration constraints, and can use model checking to ensure global properties concerning protocols, version compatibility, policies, or legal process requirements.
- *Code generation.* Once all required elementary services are implemented, stand-alone code can be generated to fully match the tracer behavior and also manage aspects like role specifications, rights, permissions, security, load balancing,

and long-running transactions management.

- **Monitoring.** Our runtime environment tracks resource consumption and availability as well as the progress of individual process instances according to their defining models.

CMDE has been successfully applied in several industrial projects, including telecommunication services, supply-chain management, bioinformatics, logistics, and healthcare. In all these cases, agility at the customer, user, and application level proved key to aligning and linking

business and IT (T. Margaria and B. Steffen, "Service Engineering: Linking Business and IT, *Computer*, Oct. 2006, pp. 45-55). We now expect an additional boost when integrating this approach into a process- or project-management environment that oversees development and evolution. This environment will include deadline management and progress reports, automatically informing all relevant parties when certain actions are required, managing different versions and product lines, and automatically steering the build and quality-management process. Perhaps not surprisingly, the development of this management

environment proved to be a prime application of CMDE. **C**

Tiziana Margaria is chair of service and software engineering at the Institute of Informatics, Potsdam University, Germany. Contact her at margaria@cs.uni-potsdam.de.

Bernhard Steffen is chair of programming systems in the Department of Computer Science, TU Dortmund University, Germany. Contact him at steffen@cs.tu-dortmund.de.

Editor: Mike Hinchey, Lero—The Irish Software Engineering Research Centre; mike.hinchey@lero.ie

ENGINEERING AND APPLYING THE INTERNET



IEEE Internet Computing magazine reports on emerging tools, technologies, and applications implemented through the Internet to support a worldwide computing environment.

UPCOMING ISSUES:

- Unwanted Traffic
- Social Computing in the Blogosphere
- Rich Internet Applications
- Information Quality in the Internet Era

The Semantics of "Semantics" • Circumventing Censorship • API Keys

www.computer.org/internet